

Reduction

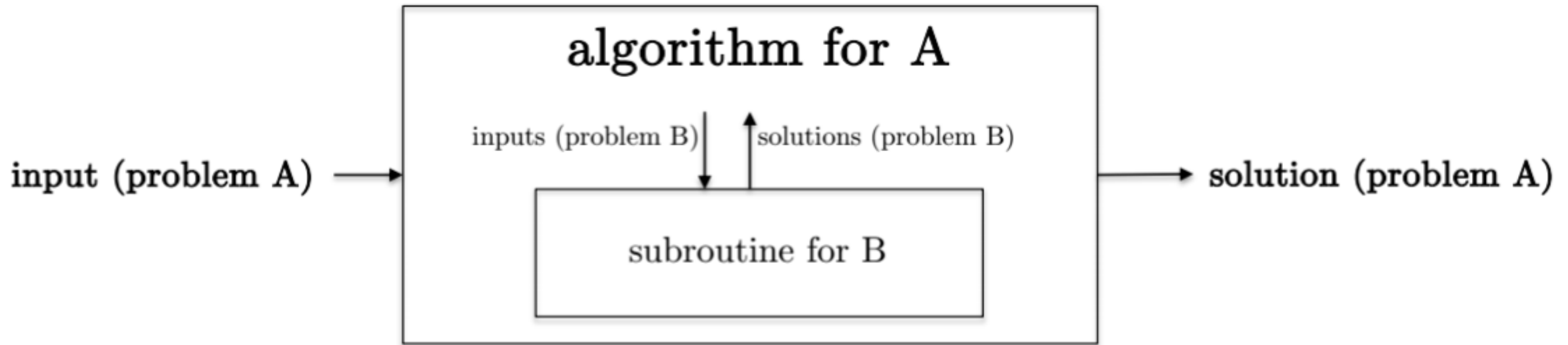
Reduction as a strategy to solve problem

- Give me 13th largest element in an unordered array A.
- Median in a list
- How to solve this problem?

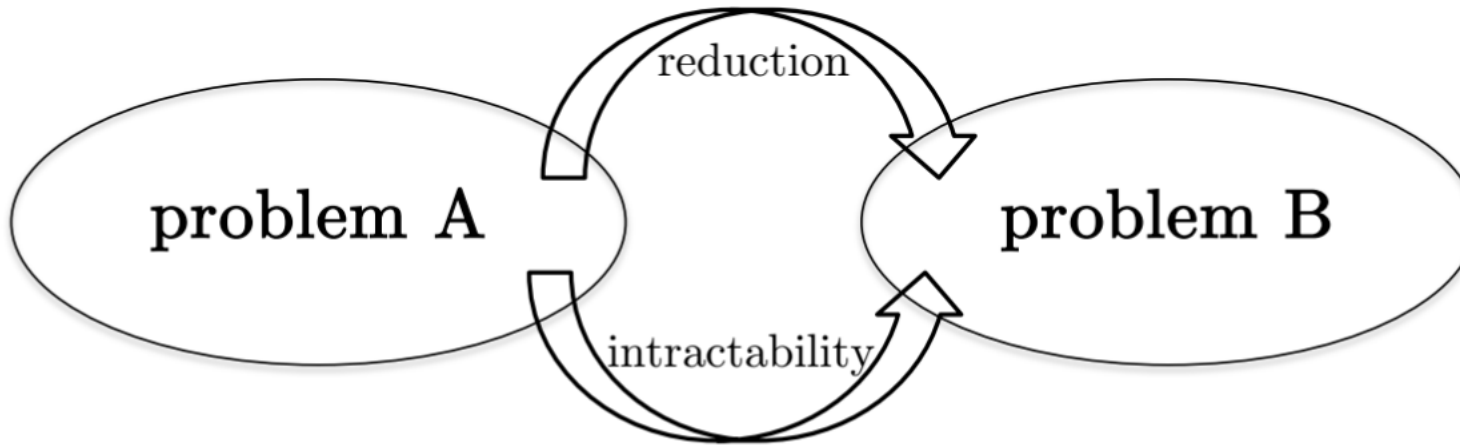
Reduction as a strategy to solve problem

- Give me 13th largest element in an unordered array A.
- How to solve this problem?
- Sort the list in descending order, return A[12]

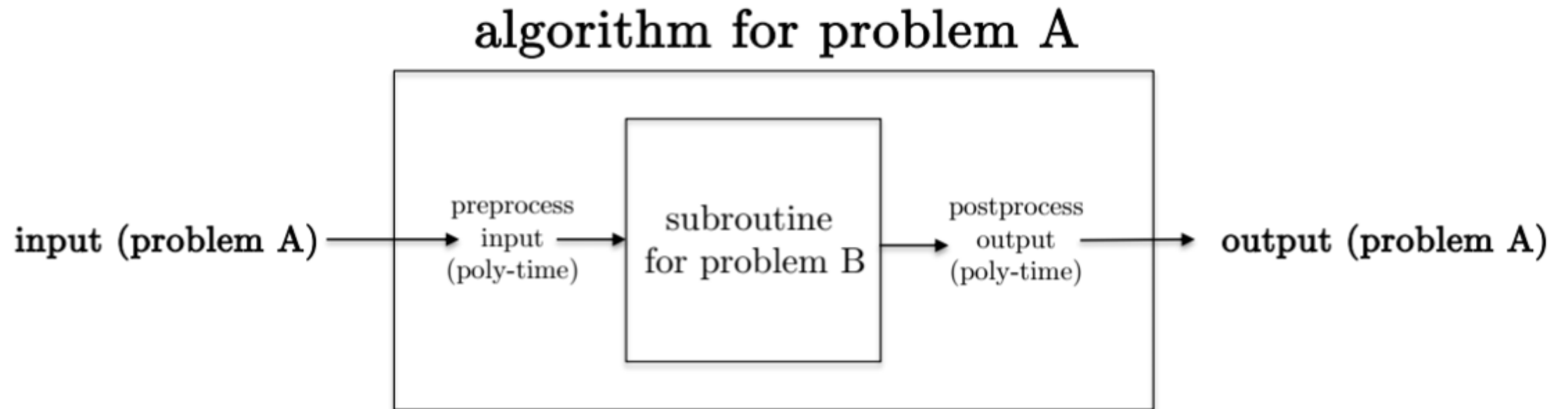
Reduction: Problem A $\rightarrow$ Problem B



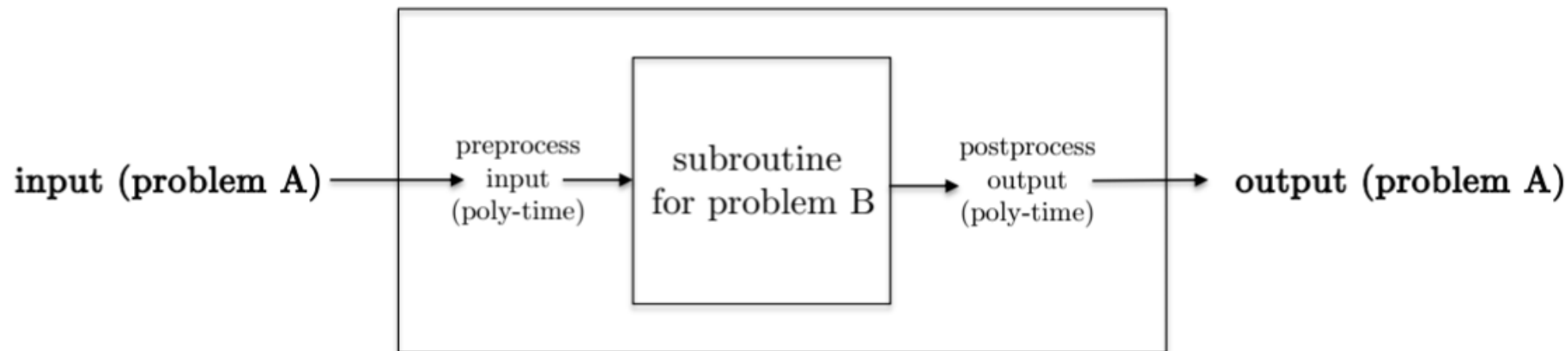
Show problem B is intractable



Using Reduction



algorithm for problem A



Simplest-Imaginable Reduction from A to B

1. *Preprocessor*: Given an instance of problem A , transform it in polynomial time to an instance of problem B .
2. *Subroutine*: Invoke the assumed subroutine for B .
3. *Postprocessor*: Transform the subroutine's output in polynomial time to a correct output for the given instance of A .

How to prove Y is NP-Complete

- Show that Y is in NP:
Verification steps for correctness should be of polynomial time
- Find a problem X with a known proof of hardness or NP-completeness
- Reduce the problem X to Y
- The reduction should be of polynomial time (fast)

Two examples of reduction

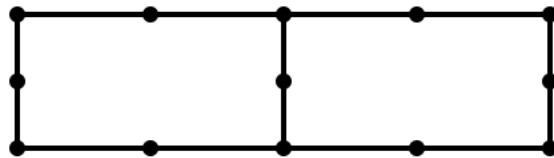
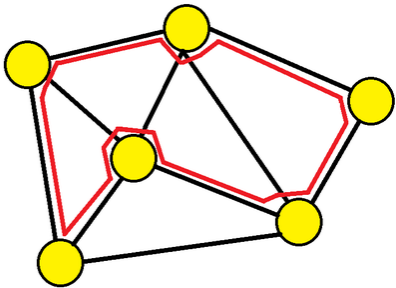
- Hamiltonian Cycle to Travelling Salesman Problem
- 3-SAT to Independent set in graph

Hamiltonian Cycle to Travelling Salesman Problem

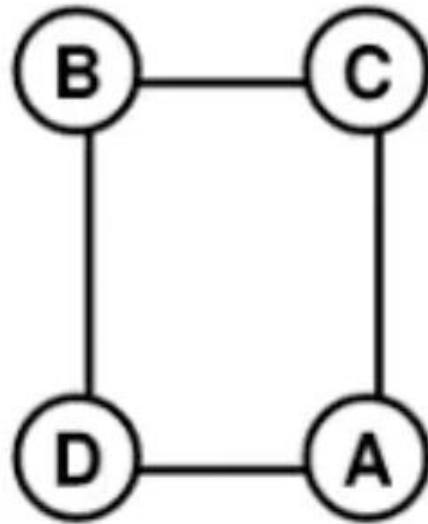
- Reduction

Relationship between Hamiltonian and TSP

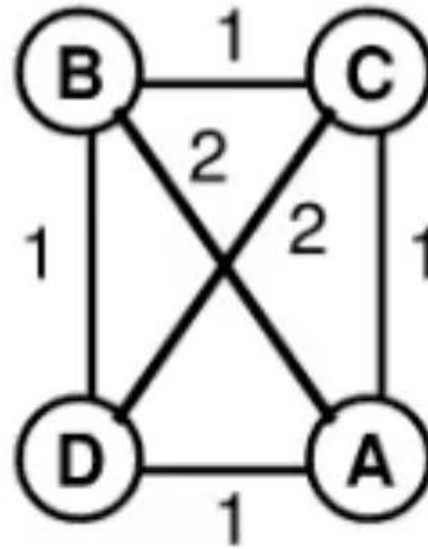
Hamiltonian cycle in Graph	Travelling Salesman Problem
Find paths through a given graph such that those paths visit each node exactly once after the start, and end where they begin/end. In some graphs, Hamiltonian cycle are not possible	Find the minimum cost Hamiltonian cycle on a weighted graph.



G has a hamiltonian cycle

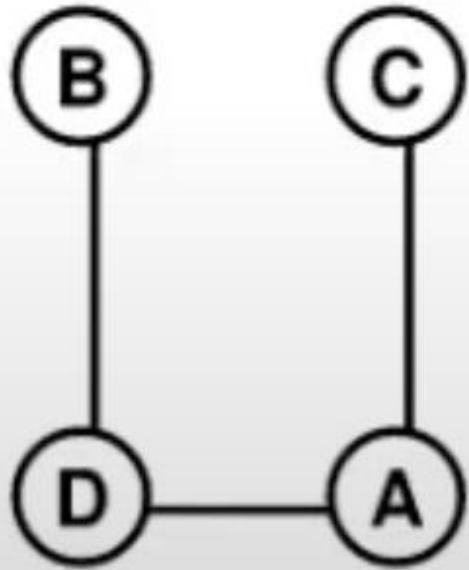


Graph G

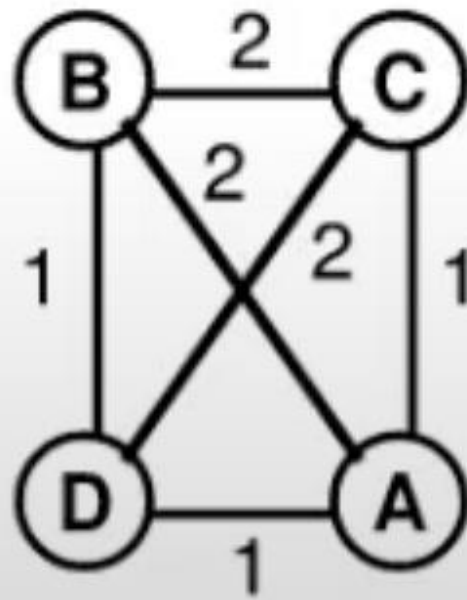


Graph G*

No Hamiltonian cycle in G



Graph G



Graph G^*

Polynomial time verification

For Hamiltonian Cycle

The class NP consists of those problems that are “verifiable” in polynomial time. What do we mean by a problem being verifiable? If we were somehow given a “certificate” of a solution, then we could verify that the certificate is correct in time polynomial in the size of the input to the problem. For example, in the hamiltonian-cycle problem, given a directed graph $G = (V, E)$, a certificate would be a sequence $\langle v_1, v_2, v_3, \dots, v_{|V|} \rangle$ of $|V|$ vertices. We could easily check in polynomial time that $(v_i, v_{i+1}) \in E$ for $i = 1, 2, 3, \dots, |V| - 1$ and that $(v_{|V|}, v_1) \in E$ as well.

Travelling Salesman proof by Reduction

- Hamiltonian cycle problem is similar to TSP
- Hamiltonian cycle problem is NP-complete
- Reduce Hamiltonian cycle to TSP
- Use the solution of TSP to answer HC problem
- Verify the solution

Travelling Salesman is in class NP

Polynomial time verification

- Input: A sequence of n vertices in the tour.
- Verification
 - Check that input sequence do not have duplicates
 - Sum up the edge costs/weights
 - Is the sum at most K ?

Reducing TSP to Hamiltonian Cycle

To prove that TSP is NP-hard, we show that $\text{HAM-CYCLE} \leq_P \text{TSP}$. Let $G = (V, E)$ be an instance of HAM-CYCLE. We construct an instance of TSP as follows. We form the complete graph $G' = (V, E')$, where $E' = \{(i, j) : i, j \in V \text{ and } i \neq j\}$, and we define the cost function c by

$$c(i, j) = \begin{cases} 0 & \text{if } (i, j) \in E, \\ 1 & \text{if } (i, j) \notin E. \end{cases}$$

(Note that because G is undirected, it has no self-loops, and so $c(v, v) = 1$ for all vertices $v \in V$.) The instance of TSP is then $\langle G', c, 0 \rangle$, which we can easily create in polynomial time.

We now show that graph G has a hamiltonian cycle if and only if graph G' has a tour of cost at most 0. Suppose that graph G has a hamiltonian cycle h . Each edge in h belongs to E and thus has cost 0 in G' . Thus, h is a tour in G' with cost 0. Conversely, suppose that graph G' has a tour h' of cost at most 0. Since the costs of the edges in E' are 0 and 1, the cost of tour h' is exactly 0 and each edge on the tour must have cost 0. Therefore, h' contains only edges in E . We conclude that h' is a hamiltonian cycle in graph G . ■

NP-completeness proof for Travelling Salesman problem

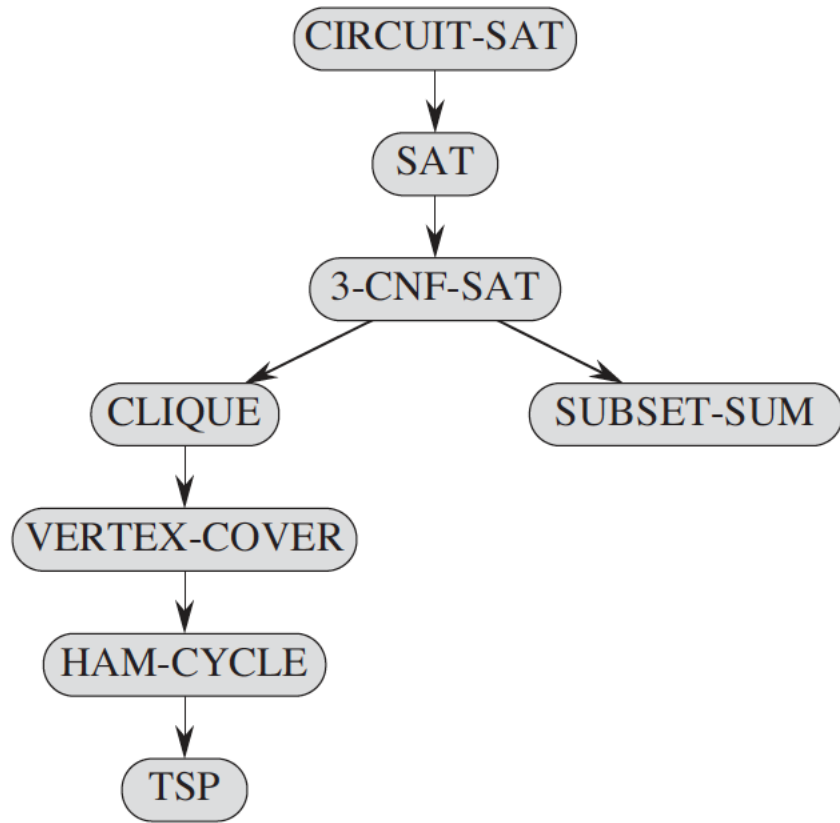


Figure 34.13 The structure of NP-completeness proofs in Sections 34.4 and 34.5. All proofs ultimately follow by reduction from the NP-completeness of CIRCUI-T-SAT.

3-SAT problem

Boolean expression made of:

- 3 clauses C_1, C_2, C_3
- 3 variables x_1, x_2, x_3 in $\{\text{True/False}\}$

$$\underbrace{x_1 \vee x_2 \vee x_3}_{C_1}$$

$$\underbrace{\neg x_1 \vee x_2 \vee x_3}_{C_2}$$

$$\underbrace{\neg x_1 \vee \neg x_2 \vee \neg x_3}_{C_3}$$

3-SAT problem

Boolean expression made of:

- 3 clauses C_1, C_2, C_3
- 3 variables x_1, x_2, x_3
- $\vee$ is OR, $\neg$ is NOT

$$\underbrace{x_1 \vee x_2 \vee x_3}_{C_1}$$

$$\underbrace{\neg x_1 \vee x_2 \vee x_3}_{C_2}$$

$$\underbrace{\neg x_1 \vee \neg x_2 \vee \neg x_3}_{C_3}$$

- Is the Boolean expression satisfied?
Is $C_1 \ \&\& \ C_2 \ \&\& \ C_3 = \text{True}$?

Problem: 3-SAT

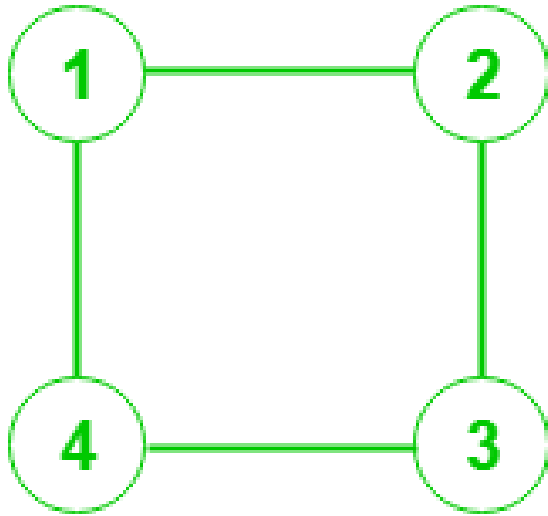
Input: A list of Boolean decision variables $x_1, x_2, \dots, x_n$; and a list of constraints, each a disjunction of at most three literals.

Output: A truth assignment to $x_1, x_2, \dots, x_n$ that satisfies every constraint, or a correct declaration that no such truth assignment exists.

Can this be 3-SAT be satisfied?

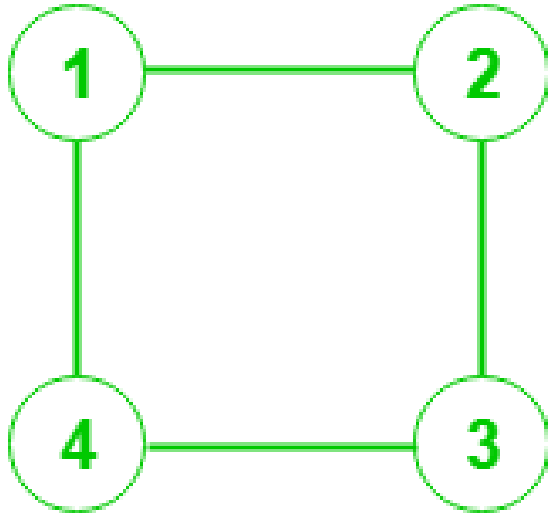
$$\begin{array}{cccc} x_1 \vee x_2 \vee x_3 & x_1 \vee \neg x_2 \vee x_3 & \neg x_1 \vee \neg x_2 \vee x_3 & x_1 \vee \neg x_2 \vee \neg x_3 \\ \neg x_1 \vee x_2 \vee x_3 & x_1 \vee x_2 \vee \neg x_3 & \neg x_1 \vee x_2 \vee \neg x_3 & \neg x_1 \vee \neg x_2 \vee \neg x_3, \end{array}$$

Independent Set problem in Graph



- Select non-adjacent vertices
- Independent set = $\{1, 3\}$ and $\{2, 4\}$

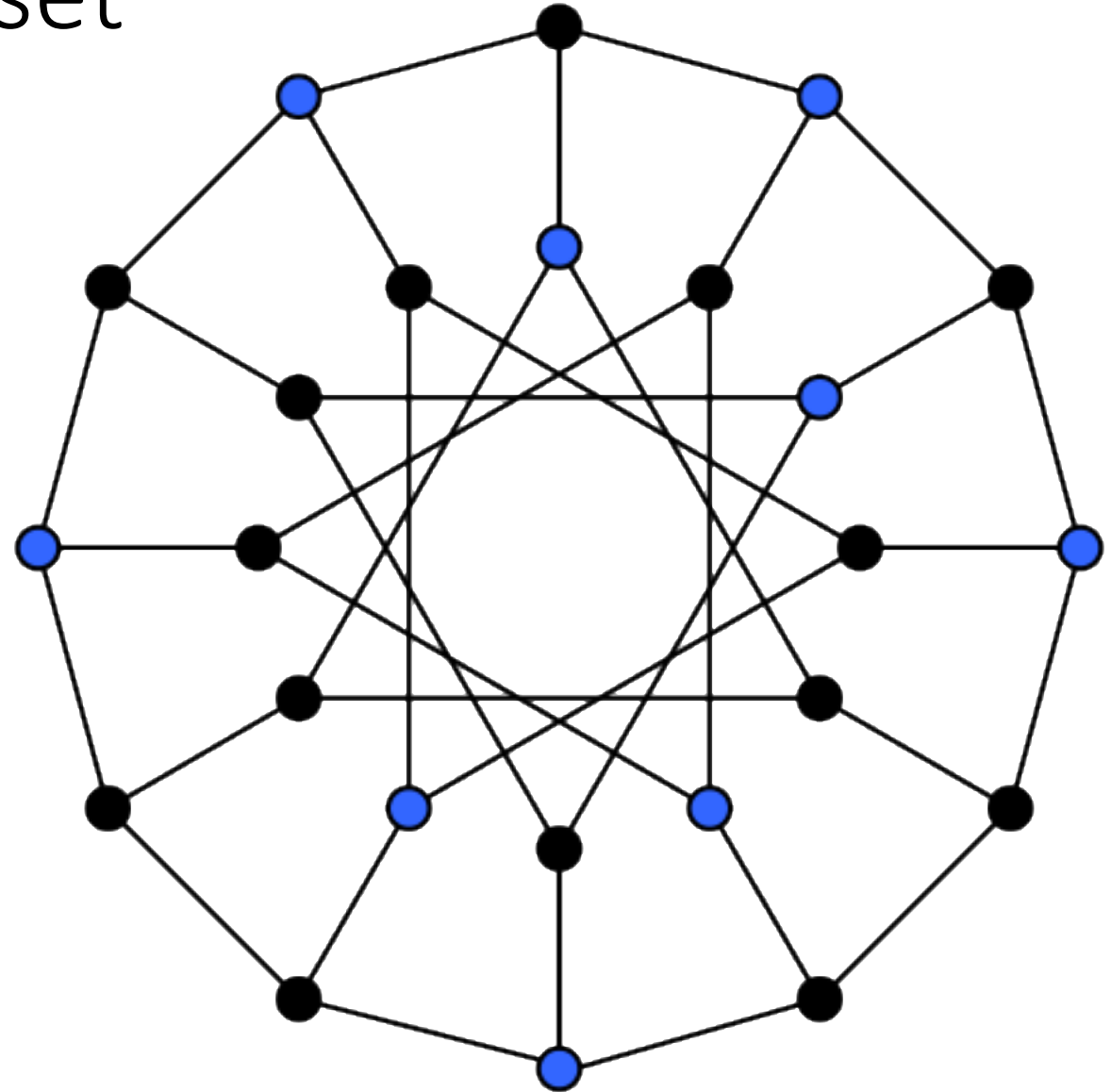
Independent Set problem in Graph



- Independent set = $\{1, 3\}$ and $\{2, 4\}$
- Maximum independent set version.

Maximum Independent set

The nine blue vertices form
a maximum independent set
for the graph



3-SAT is NP-hard

- Was proved in 1970s
- Won a Turing prize in CS – Cook and Karp

Using 3-SAT

- Prove that Independent Set is NP-Hard
- Reducing 3-SAT to Independent Set

Input of 3-SAT => Vertices of a graph

$$\underbrace{x_1 \vee x_2 \vee x_3}_{C_1}$$

$$\underbrace{\neg x_1 \vee x_2 \vee x_3}_{C_2}$$

$$\underbrace{\neg x_1 \vee \neg x_2 \vee \neg x_3}_{C_3}$$

$$\begin{array}{c} (C_1) \\ x_1 = T \end{array}$$

$$\begin{array}{c} (C_1) \\ x_2 = T \end{array}$$

$$\begin{array}{c} (C_1) \\ x_3 = T \end{array}$$

$$\begin{array}{c} (C_2) \\ x_1 = F \end{array}$$

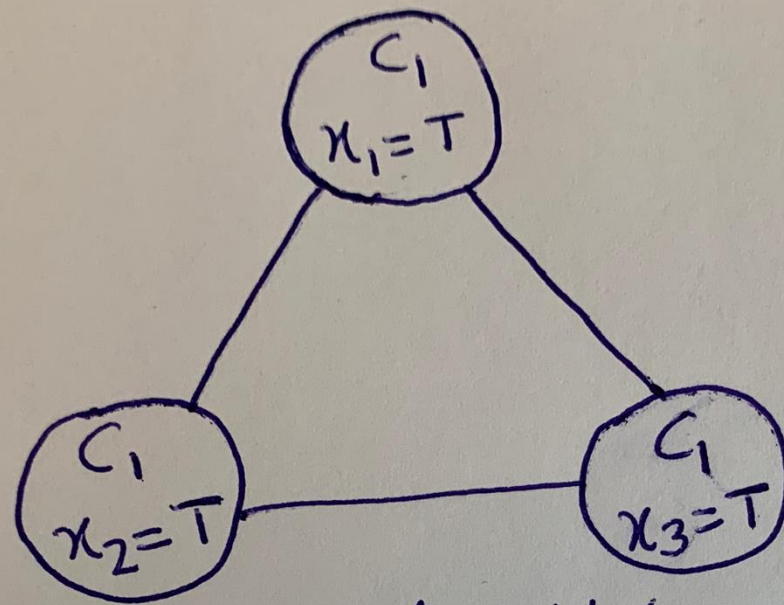
$$\begin{array}{c} (C_2) \\ x_2 = T \end{array}$$

$$\begin{array}{c} (C_2) \\ x_3 = T \end{array}$$

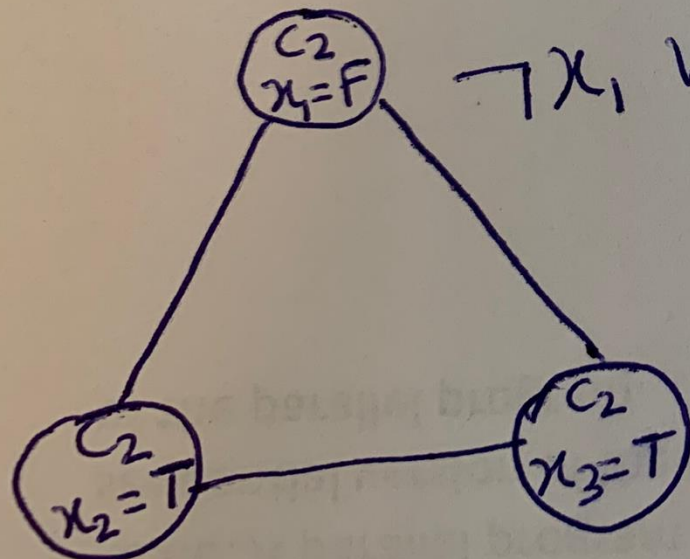
$$\begin{array}{c} (C_3) \\ x_1 = F \end{array}$$

$$\begin{array}{c} (C_3) \\ x_2 = F \end{array}$$

$$\begin{array}{c} (C_3) \\ x_3 = F \end{array}$$

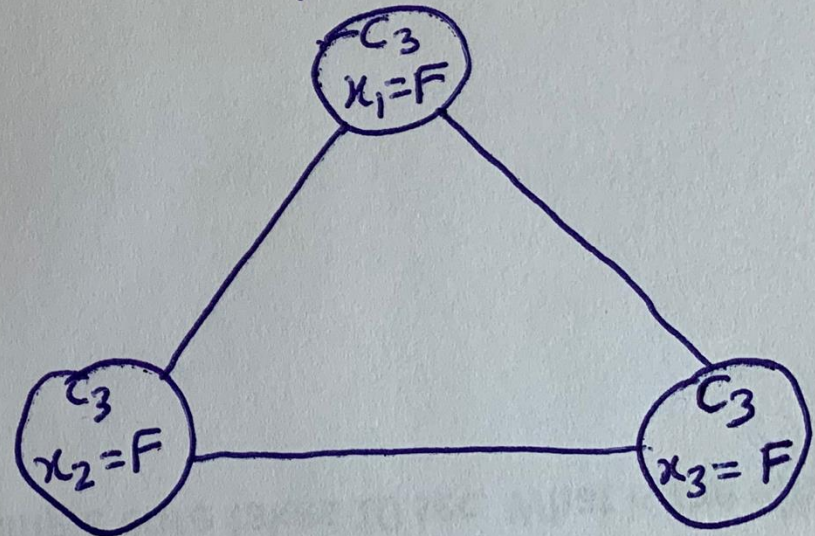


$$x_1 \vee x_2 \vee x_3$$

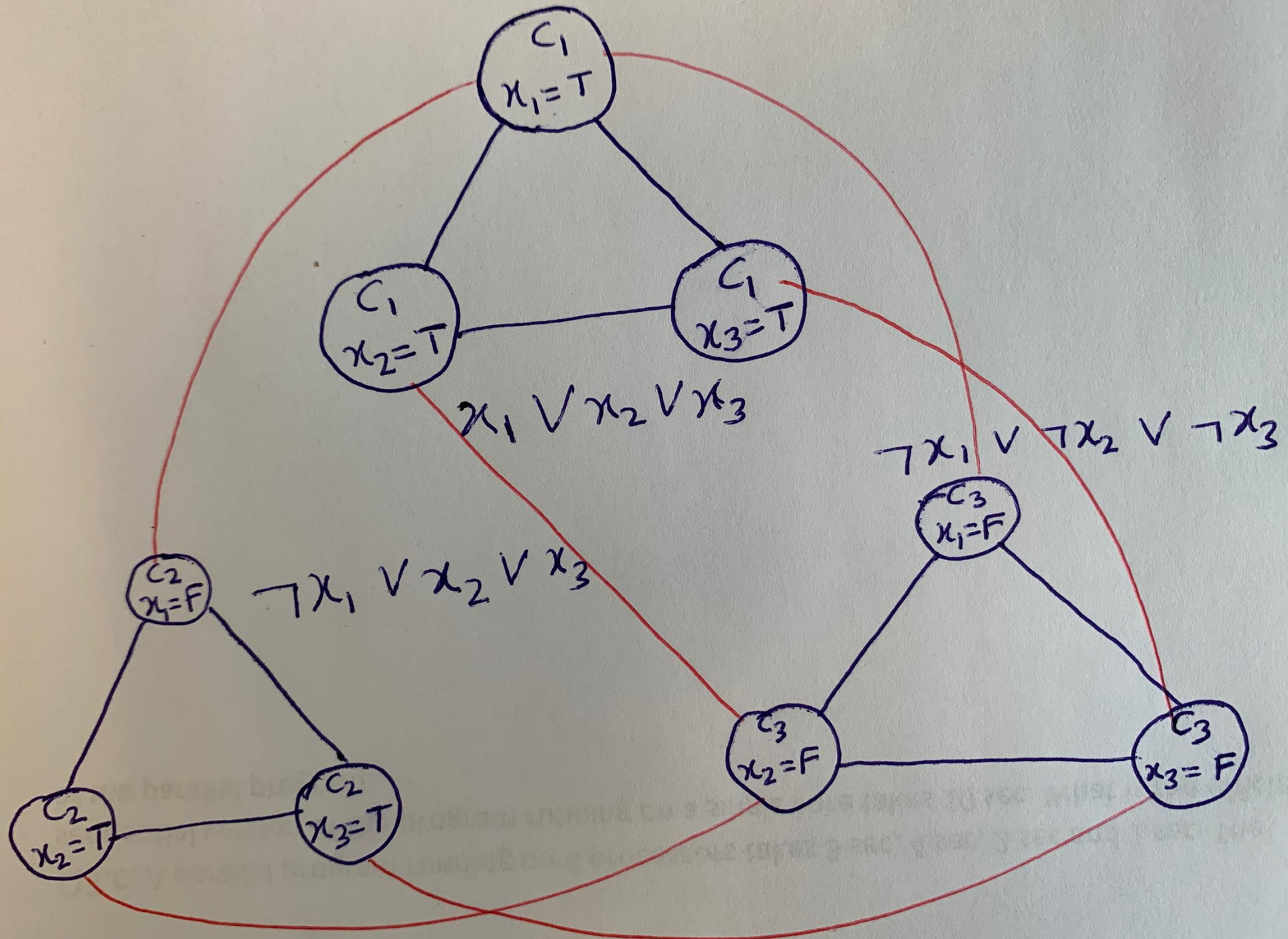


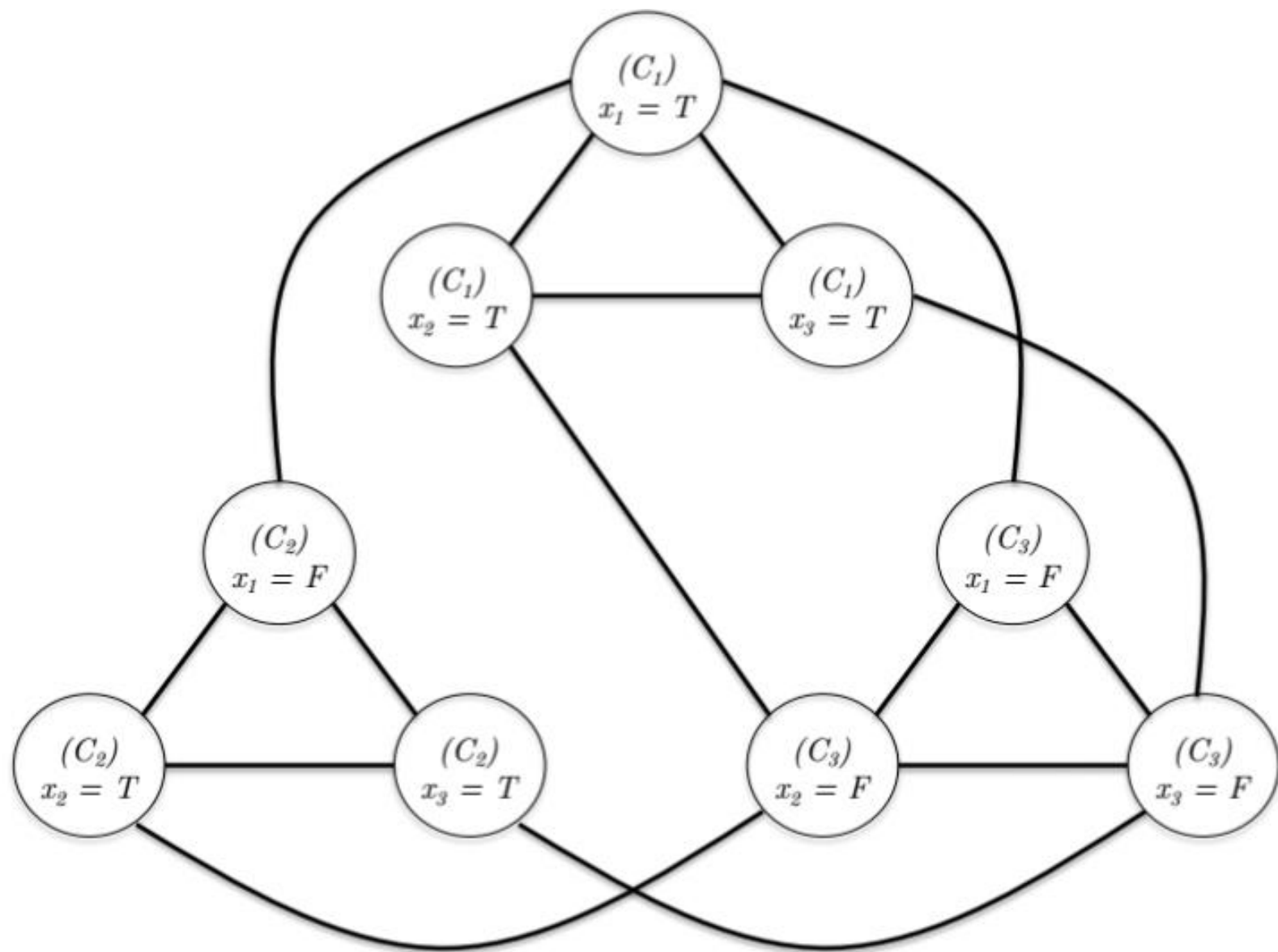
$$\neg x_1 \vee x_2 \vee x_3$$

$$\neg x_1 \vee \neg x_2 \vee \neg x_3$$



Red edges between
any two clauses
where
truth assignment
differs
 $x_1 = T$ in clause 1
 $x_1 = F$ in clause 2





Clause 1

$x_2 = T$

Clause 2

$x_1 = F$

Clause 3

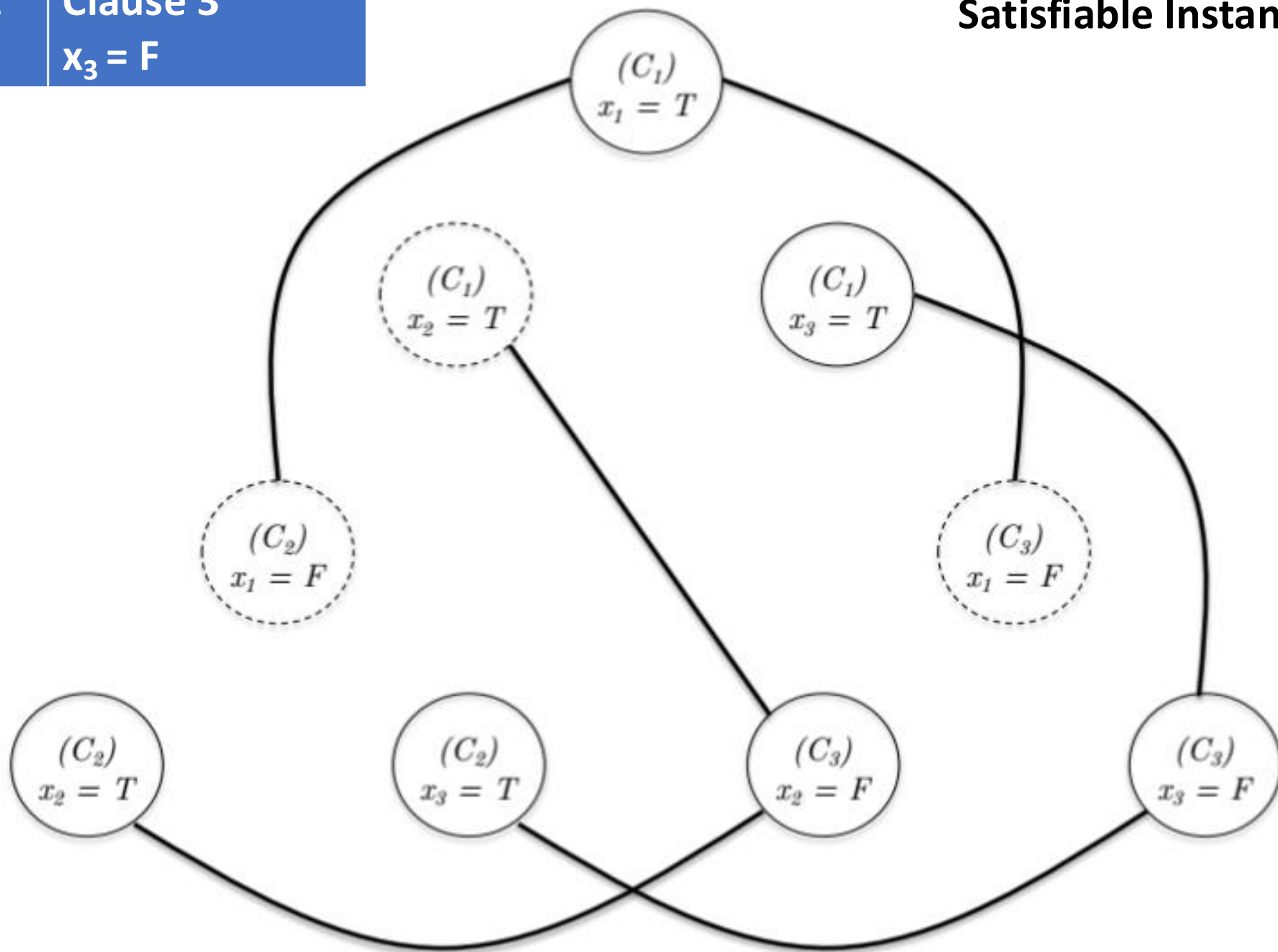
$x_3 = F$

Satisfiable Instance

$$\underbrace{x_1 \vee x_2 \vee x_3}_{C_1}$$

$$\underbrace{\neg x_1 \vee x_2 \vee x_3}_{C_2}$$

$$\underbrace{\neg x_1 \vee \neg x_2 \vee \neg x_3}_{C_3}$$



Description of Reduction

- Preprocessor
- Postprocessor

Description of Reduction

Preprocessor. Given an arbitrary 3-SAT instance, with n variables and m constraints with at most three literals each, the preprocessor constructs a corresponding graph $G = (V, E)$. It defines $V = V_1 \cup V_2 \cup \dots \cup V_m$, where V_j is a group with one vertex per literal of the j th constraint. It defines $E = E_1 \cup E_2$, where E_1 contains one edge per pair of vertices that reside in the same group and E_2 contains one edge per pair of conflicting vertices (corresponding to requests for opposite assignments to the same variable).

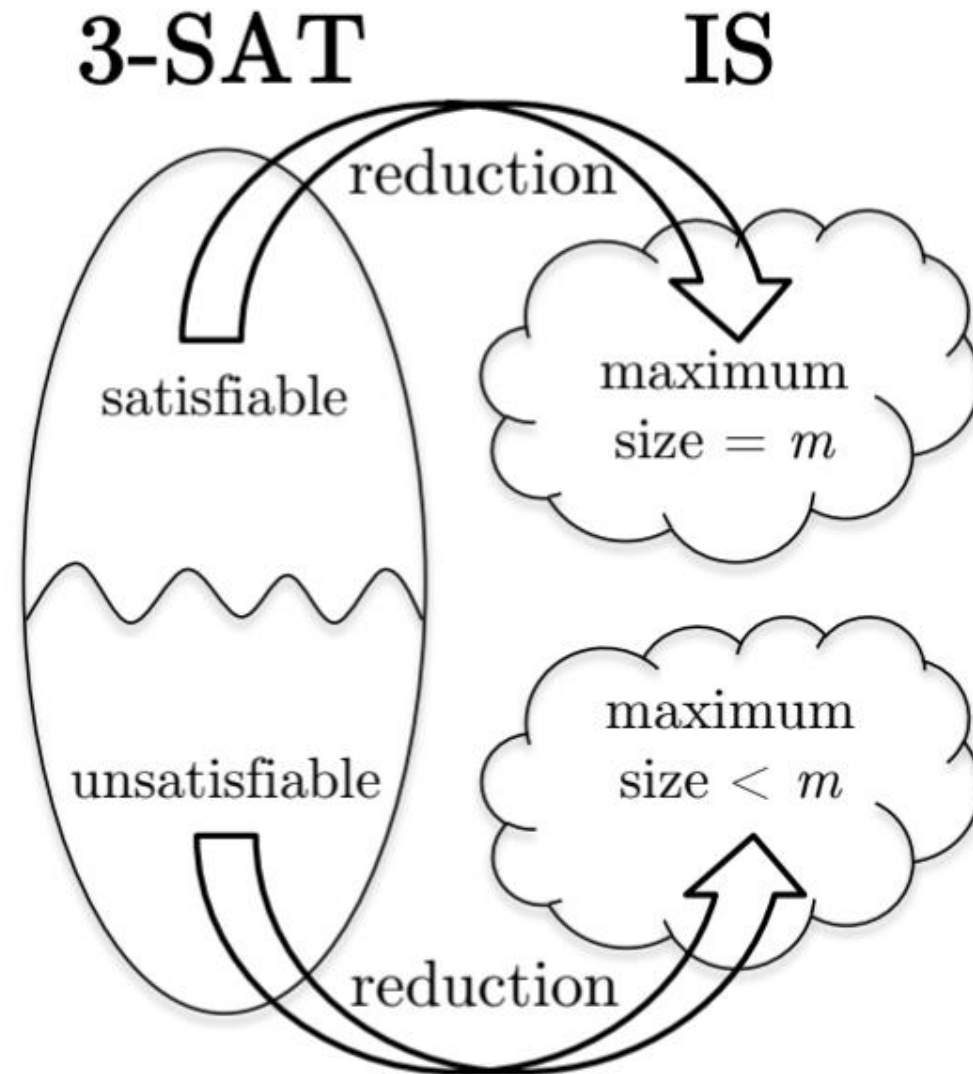
Description of Reduction

Postprocessor. If the assumed subroutine returns an independent set of the graph G constructed by the preprocessor with at least m vertices, the postprocessor returns an arbitrary truth assignment consistent with the corresponding variable assignment requests. Otherwise, the postprocessor returns “no solution.”

Proof of correctness

Preprocessor translates
an instance of 3-SAT to IS

Max **size** of independent set
m clauses/constraints



Property

- NP-complete problems are reducible to each other
- Polynomial-time reductions provide a formal means for showing that one problem is at least as hard as another
- If any NP-complete problem can be solved in polynomial time, then every problem in NP has a polynomial-time solution

Reduction

- One NP complete problem reduces to another NP complete problem
- If we solve one problem in class NP-complete, we can solve all problems in this class
- Basis for proof of problem's difficulty